

指令的执行状态，从而对程序中定义的条件进行判断，以决定接下来是否进行跳转。在进行运算时，程序需要先将数据从内存加载到通用寄存器中，运算完成后再将结果写回内存。在进行函数调用时，调用者可通过通用寄存器和内存将参数传递给被调用者；被调用者在运行时所使用的本地变量以及函数返回的地址被保存在运行时栈中，处理器通过栈寄存器来保存当前栈顶的位置。

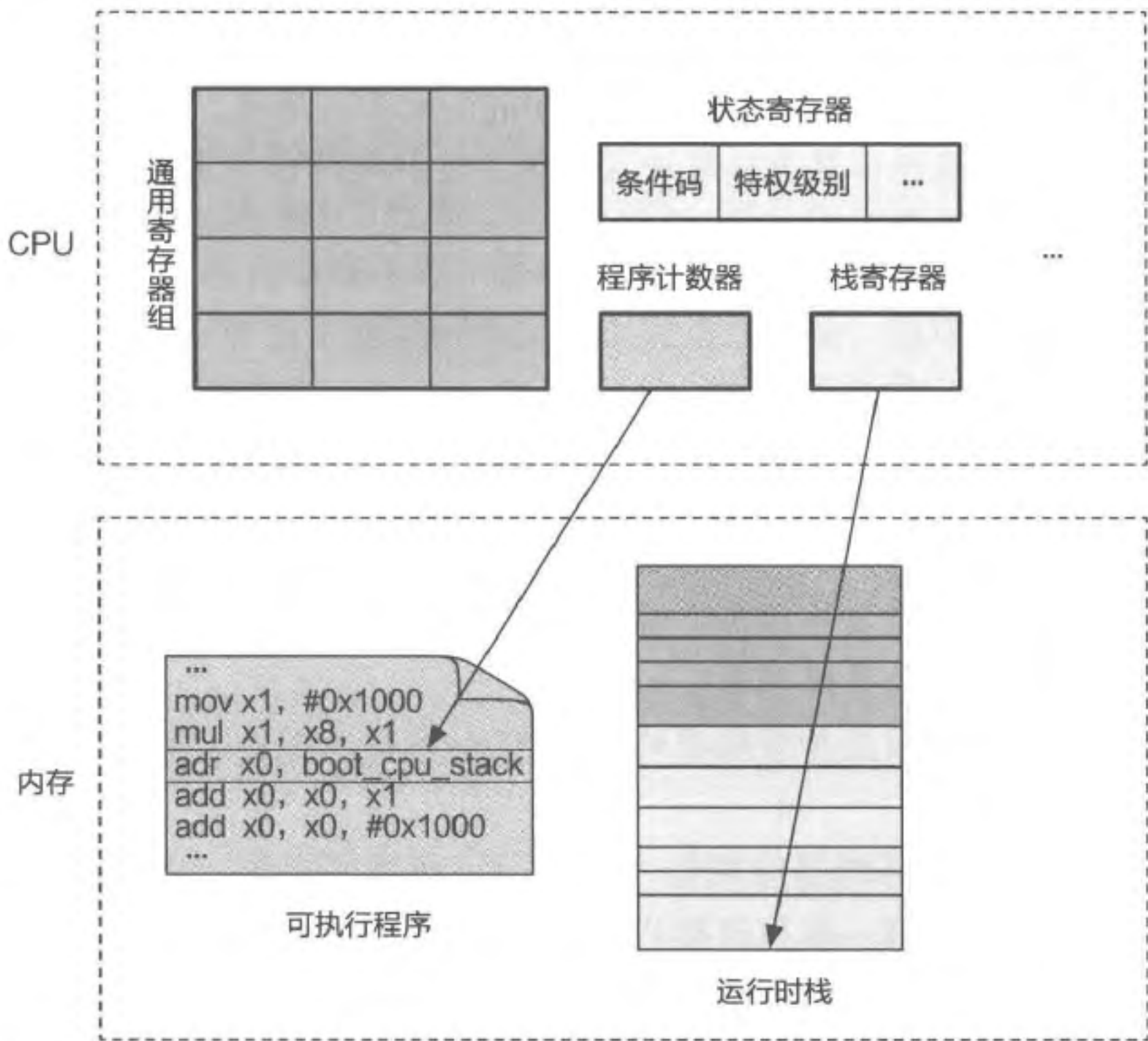


图 3.9 应用程序依赖的处理器状态，包括通用寄存器组、状态寄存器、程序计数器和栈寄存器。其中，程序计数器和栈寄存器分别存放内存中机器指令和运行时栈的地址

应用程序在运行的过程中，由于某些原因（比如时间片运行结束，需要访问硬件设备等）会切换到操作系统执行，那么在操作系统开始执行前，这些在处理器中属于应用程序的状态必须保存起来，等操作系统运行结束再恢复，从而继续应用程序的运行。我们将会在下一节探讨进程的概念时对此进行更深入的阐释。

小知识：栈的增长方向

虽然运行时栈的增长方向因架构和编译器而异，但大部分情况下我们见到的栈都从高地址向低地址增长，这种设计其实是为了充分利用内存空间。应用程序能够动态分配的内存主要分为两类：栈内存和堆内存。为了让栈和堆可以共享同一个大的内存池，操作系统往往会让栈与堆从一块内存区域的两端向中间增长。其中，堆从低地址

向高地址增长，而栈从高地址向低地址增长。

这样的设计事实上带来了严重的安全问题。如果应用程序在栈上分配了一块缓冲区并向其写入数据，一旦写入的数据量超过了缓冲区的大小，超出的部分就会覆盖栈上的其他数据，即缓冲区溢出（Buffer Overflow）。由于栈的增长方向是从高到低的，与缓冲区溢出的方向相反，因此攻击者可以通过溢出来覆盖返回地址（因为当前函数的返回地址会被保存在比缓冲区更高的地址），使函数在返回时跳转到恶意代码处，实现控制流劫持（Return-Oriented Programming）攻击。反之，如果栈的增长方向是从低到高的，返回地址的位置会比缓冲区更低，这样缓冲区就算溢出也无法覆盖返回地址的值，使攻击的难度大大增加。

3.2 操作系统的硬件运行环境

本节主要知识点

- 为什么要区分用户态与内核态？两者的运行环境有什么区别？
- 什么是中断、异常和系统调用？
- 用户态和内核态之间是如何切换的？CPU 和内核分别做了哪些操作？

操作系统的硬件运行环境是应用程序硬件运行环境的超集：除了运算、访存、函数调用等基本功能外，还包括一些只有操作系统才有权限执行的功能，比如对 CPU 中断的开关、对频率的调整、对内存的配置、对设备的操作等。这些功能之所以只允许操作系统使用，是因为它们会影响系统全局的状态而不仅仅是某一个应用程序的状态。为了区分操作系统与应用程序的不同运行环境，现代 CPU 通常会提供不同的特权级，让用户程序运行在低特权级，操作系统运行在更高的特权级，并限制只有高特权级才能执行特权操作。同时，CPU 也提供了在低特权级与高特权级之间的切换机制。

3.2.1 特权级别与系统 ISA

设想一下，如果不区分应用程序和操作系统运行的级别，换句话说，就是允许应用程序和操作系统一样能够使用硬件提供的所有功能、控制整个计算机，会发生什么呢？让我们考虑有多个应用程序同时运行的情况。在这些应用程序彼此配合良好且运行正常的情况下，即使不区分特权级别，系统也是可以正常运行的；但如果应用程序出现错误，则有可能影响其他应用甚至整个系统。例如，某个应用程序错误地运行了 CPU 的重置指令（reset），导致所有应用的状态丢失；两个应用配合失误，同时向内存或磁盘的同一个位置写入数据，则会导致其中一份数据的丢失。此外，由于应用程序有权限访问整个

内存，如果运行了一个恶意应用程序，则该应用可以随意窃取或篡改其他应用程序在内存中的数据。可以看到，允许任意应用程序直接访问所有的硬件资源是非常危险的。实际上，本书第2章提到的简要结构就没有区分特权级别，通常用于一些简单场景（如嵌入式设备），但其应用场景相当受限。

为了区分应用程序和操作系统的运行权限，CPU 为两者提供了不同的特权级别：用户态（User-mode）和内核态（Kernel-mode）。ISA 作为 CPU 向软件提供的接口，也对应地分为用户 ISA 与系统 ISA。在用户态运行的软件只能使用用户 ISA，在内核态运行的软件则可以同时使用系统 ISA 和用户 ISA。

3.1 节介绍的 ISA 包括通用寄存器、栈寄存器、条件码寄存器、运算指令等，都属于用户 ISA，系统 ISA 则包含系统状态、系统寄存器与系统指令。其中，系统状态包括当前 CPU 的特权级别、CPU 发生错误时引发错误的指令地址、程序运行状态等。存储这些状态的寄存器称为系统寄存器（System Register），这些寄存器只能由运行在内核态的软件通过系统指令来访问。

运行在内核态的操作系统可通过系统 ISA 来管理系统的硬件以及应用程序。当运行在用户态的应用程序需要操作外部设备（比如向屏幕输出“hello, world”）时，需要向操作系统提出申请，由其代为完成。如果应用程序试图强行在用户态执行系统 ISA 中独有的系统指令，处理器会制止这次操作，并切换到内核态，由操作系统来决定应该如何处理这次越权操作（通常是中止该应用）。

小知识

操作系统并不一定完全运行在内核态。UNIX 就把操作系统分为“内核”与“外壳”（shell）两部分，其中内核运行在内核态，shell 运行在用户态并通过命令行与用户交互。由于内核态拥有最高控制权，一旦内核存在 bug 或被攻击者控制，其后果往往非常严重。因此，操作系统的设计者往往选择将部分功能放到用户态，从而减少内核的代码。第2章介绍的微内核架构就是一个很好的例子。在本章中并不刻意区分“操作系统”和“操作系统内核”。

案例分析：ARM 的特权级别和系统 ISA

真实的 CPU 并不一定只有用户态和内核态两种特权级别。我们以 AArch64 为例介绍 CPU 特权级别和系统 ISA 的一种具体实现。如图 3.10 所示，AArch64 中的特权级别被称为异常级别（Exception Level, EL），共分为 4 个级别，其中应用程序运行在 EL0，操作系统运行在 EL1。CPU 当前的特权级状态保存在 PSTATE 寄存器中，当 CPU 发生状态切换的时候，该寄存器中的状态会随之更新。4 个级别具体如下：

- EL0：用户态，应用程序通常运行在该特权级别。
- EL1：内核态，操作系统通常运行在该特权级别。

- EL2：用于虚拟化场景，虚拟机监控器（Virtual Machine Monitor, VMM，也称为 Hypervisor）通常运行在该特权级别。
- EL3：与安全特性 TrustZone 相关，负责普通世界（Normal World）和安全世界（Secure World）之间的切换，本章主要集中在普通世界。

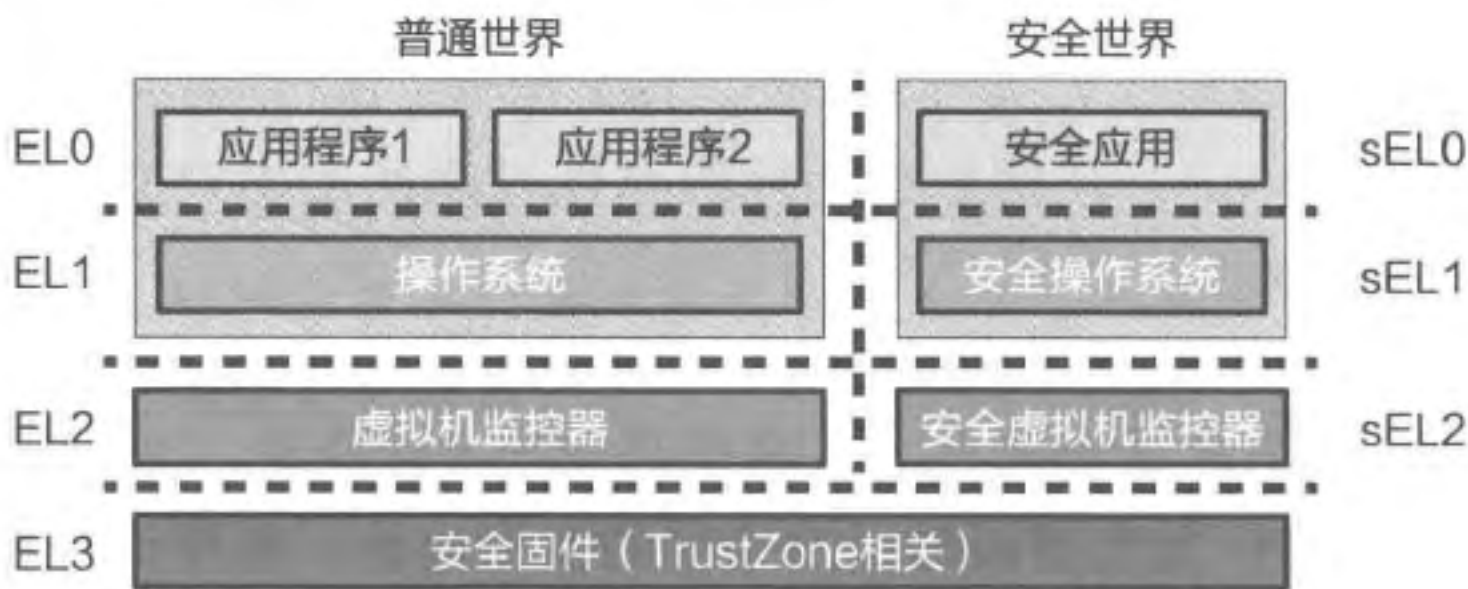


图 3.10 AArch64 架构下 CPU 的 4 个特权级别^[1]

小知识

TrustZone 是从 ARMv6 架构开始引入的安全特性，如今已被广泛使用。该特性从逻辑上将整个系统分为安全世界和普通世界，计算资源可以被划分到这两个世界。安全世界可以不受限制地访问所有的计算资源，而普通世界不能访问被划分到安全世界的计算资源。EL2 的引入则是为了支持比操作系统权限更高的虚拟机监控器的运行，第 14 章将对其进行介绍。

需要注意的是，对于许多 ISA 来说，当 CPU 在内核态运行用户 ISA（比如函数调用）时，一般会使用用户 ISA 的寄存器（比如 SP）。这也是为什么从用户态切换到内核态时，首先需要将用户态寄存器的值保存到内存。AArch64 则采用了略微不同的方法：为一些常用的用户态寄存器在不同特权级提供不同的硬件副本。例如，对于栈寄存器 SP，AArch64 提供了 SP_EL0（用户态）与 SP_EL1（内核态）。其中，用户态在函数调用时使用 SP_EL0，无法访问 SP_EL1；内核态则使用 SP_EL1，但也有权限读写 SP_EL0。这么设计的好处在于，当 CPU 从用户态切换到内核态时，内核代码无须保存 SP_EL0 寄存器中的值，在切换回用户态时也无须恢复，从而可以降低切换的时延。

AArch64 的系统寄存器负责保存硬件系统状态，以及为操作系统提供管理硬件的接口。系统 ISA 提供了 `mrs` 和 `msr` 两条特权指令[⊖]，其作用是从系统寄存器中读取值（获取系统信息）或向系统寄存器中写入值（控制系统状态）。系统 ISA 的指令只有在特权态

⊖ `mrs` 和 `msr` 常常容易混淆，只需要记住 `mrs` 的全称是 Move to Register from State register，目标在前源在后，顺序与 `memcpy` 的参数是一样的。

才能运行，CPU 在执行相关指令前会先根据 PSTATE 中的状态来判断是否合法。例如，当 PSTATE 记录当前运行级别为 EL0 时，CPU 运行的指令无权访问 ELR_EL1 系统寄存器。由于 AArch64 有多个特权级，因此对于系统寄存器，也需要通过类似的后缀来表明这些寄存器在哪一个特权级下使用，例如 TTBR0_EL1（一阶段页表基地址寄存器）和 TTBR0_EL2（用于虚拟化的二阶段页表基地址寄存器）。表 3.4 总结了 AArch64 中的常用寄存器在 EL0 和 EL1 特权级下的可见情况。我们将在下一节介绍系统寄存器的用途。

表 3.4 AArch64 中的常用寄存器在不同特权级下的可见情况

寄存器		EL0	EL1	描 述
通用寄存器	X0 ~ X30	✓	✓	
特殊寄存器	PC	✓	✓	程序计数器
	SP_EL0	✓	✓	用户态栈寄存器
	SP_EL1		✓	内核态栈寄存器
	PSTATE	✓	✓	状态寄存器
系统寄存器	ELR_EL1		✓	异常链接寄存器
	SPSR_EL1		✓	已保存的程序状态寄存器
	VBAR_EL1		✓	异常向量表基地址寄存器
	ESR_EL1		✓	异常症状寄存器

注：PSTATE 寄存器中仅有 NZCV 四个条件码在 EL0 下可见。

3.2.2 异常机制与异常向量表

在提供多个特权级的同时，CPU 还需要提供在不同特权级之间切换的机制，本节主要关注用户态与内核态之间的切换过程。在 AArch64 平台，用户态与内核态的切换使用的是异常机制，其中，从用户态切换至内核态的过程称为陷入或下陷（Trap），从内核态切换至用户态的过程称为“从内核态返回”。为什么要称为“异常”机制呢？因为对应用程序来说，大部分时间都是在用户态执行代码，只有出现特殊情况，比如需要获得键盘输入、在显示器输出字符、被硬件产生的中断打断、发生应用程序无法处理的错误等，才需要“下陷”到内核态去处理。因此从应用程序的角度来看，下陷到内核是因为出现了“异常”情况，需要内核处理，处理完成后恢复“正常”，应用继续运行。在内核中处理异常情况的代码通常称为异常处理程序（Exception Handler）。

小知识

虽然这里说“异常”是从应用程序的角度来看的，但实际上 CPU 在内核态运行代码时也可能发生异常事件，同样会由内核的异常处理程序进行处理，唯一的区别是不需要“下陷”，即切换特权级——因为已经在内核态了。这种在异常处理时发生异常的情况通常是比较少见的，如果出现三次递归，在一些处理器平台（如 x86）则会被认为是严重错误，即“Triple-Fault”。

触发 CPU 异常机制的事件主要有三类：

- 在用户程序正常执行的过程中，CPU 可能收到一些来自外部的事件^①，如硬件时钟会周期性地发出信号。这些事件会强行打断正在运行的程序，使 CPU 下陷到内核态由操作系统处理，之后恢复用户程序，从被打断的位置继续执行，整个过程中用户程序并不会意识到自己被打断过。这类来自 CPU 外部的事件一般称为中断 (Interrupt)。
- 程序在执行中可能会遇到一些自身无法处理的问题，例如执行了一条格式错误的非法指令，或试图将数据写入只读的内存区域。这些事件同样会触发下陷，操作系统处理完成后，应用程序继续执行。若操作系统无法处理，则可强行终止程序，甚至触发重启。这类来自 CPU 内部的事件一般称为异常 (Exception)。
- 用户程序主动触发异常，向操作系统发出执行特定操作的请求，CPU 通常会为这种情况提供一条特殊的指令（如 AArch64 的 svc 指令^②）。这种情况就是系统调用 (System Call)。因为系统调用也是在 CPU 内部发生的，所以系统调用也可以被看作异常的一种。

异常机制的控制流是不受应用程序控制的：一方面，发生异常后跳转的目的地址不由应用程序控制；另一方面，这种控制流突变可能在应用程序毫无准备的情况下随时发生，图 3.11 展示了发生异常时控制流在应用与内核之间的切换过程。在异常机制中，控制流变化的特殊性还体现在如何返回用户态上。在用户态发生函数调用时，被调用者返回后，会执行调用代码之后的下一条代码。而在异常机制中，操作系统返回用户态后可能有两种情况：一种情况是执行应用程序触发下陷的指令（如系统调用 svc）之后的下一条指令，另一种情况是重新执行触发下陷的指令本身（如缺页异常，后文会详细介绍）。

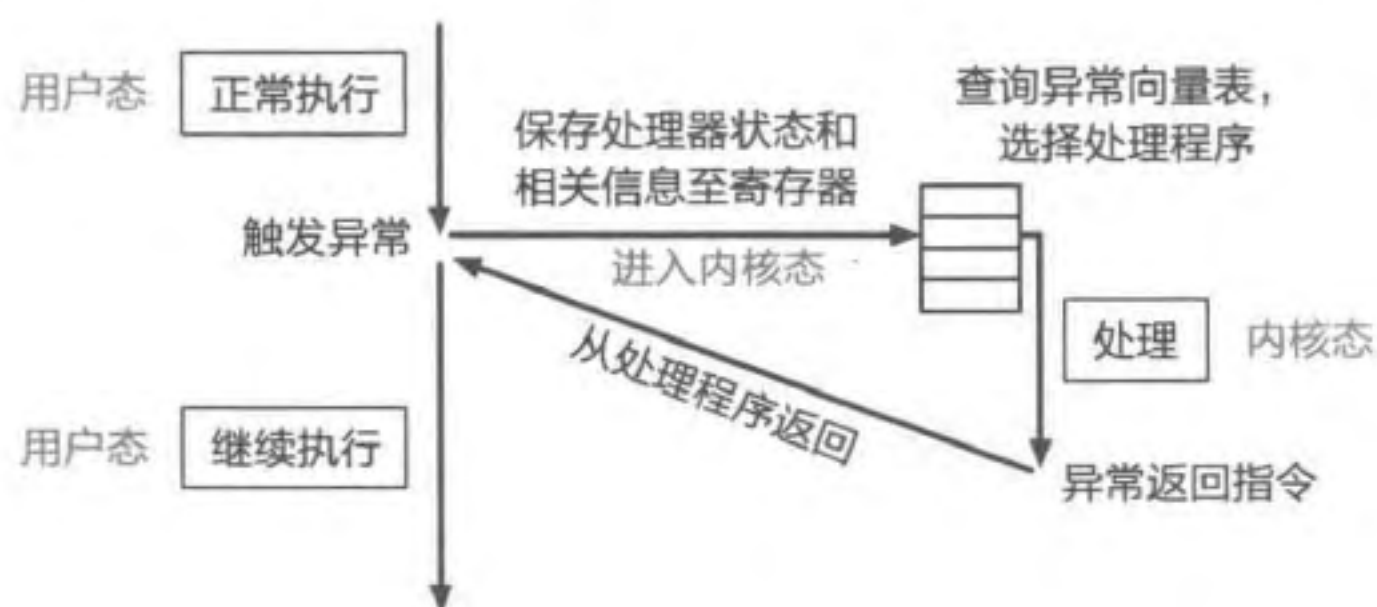


图 3.11 异常控制流的示意图

当系统发生异常事件导致“下陷”到内核态时，CPU 只允许从固定的入口开始执行。为此，操作系统需要提前将代码的入口地址“告诉”处理器。对不同类型的异常事

① 外部事件是指其他设备通过 CPU 的针脚（比如中断针脚）发送信号给 CPU。

② svc 的全称是 SuperVisor Call，此处 supervisor 即为操作系统。

件，CPU 通常支持设置不同的入口。这些入口通常以一张表的形式记录在内存中，也就是异常向量表，由操作系统负责构造。在系统启动后，操作系统会将异常向量表的内存地址写入 CPU 上的一个特殊寄存器——异常向量表基地址寄存器（如 AArch64 中的 VBAR_EL1 寄存器），然后开启中断，这样便完成了异常机制的初始化。

通过上文的介绍我们可以看到，异常向量表定义的其实是操作系统的“入口”。在初始化完成之后，操作系统之所以需要运行，是因为发生了异常事件需要处理。换句话说，如果应用程序运行一切“正常”，没有发生任何“异常”事件，那么操作系统就没有机会运行。在这种情况下，如果有一个应用程序运行了“while(true);”的空循环，没有任何系统调用，也不会运行出错，是不是就会独占 CPU，使操作系统永远没有机会运行呢？还好，这种情况并不会发生，因为还有中断。操作系统在运行应用程序之前，会先设置好硬件时钟以某个固定的频率（如每秒 1 000 次）产生中断，从而保证在应用运行一定时间（如 1ms）之后，一定会通过异常机制回到操作系统。通过时钟中断，操作系统牢牢地把握住了运行的主动权。但反过来，如果操作系统在关闭中断的情况下返回应用程序，很可能就“一去不复返”了。

中断与异常

不同体系结构表示中断和异常的术语有所差异。ARM 架构将中断称为“异步异常”，而将异常称为“同步异常”。这是因为中断是由硬件（如时钟）产生的，对 CPU 来说任何时候都可能发生，所以是异步；而异常是由软件产生的，对 CPU 来说，在某种条件下执行到某条指令一定会发生，所以是同步。x86 架构则采用和本书类似的术语，相对来说使用更为普遍，如表 3.5 所示。另一个不同之处在于：ARM 的“异常向量表”，x86 称之为“中断向量表”。

表 3.5 ARM 和 x86 的术语对比

本书术语	产生位置	ARM 术语	x86 术语
中断	CPU 外部	异步异常	中断
异常	CPU 内部	同步异常	异常 (Fault/Trap/Abort)

小知识

有些情况在 x86 架构中会触发异常，在 ARM 架构中则不会。例如在进行整数除法时，如果除数是零，就会引入通用寄存器无法表示的无穷大量或无穷小量，因此处理器往往会对除数为零的情况进行特别处理。在 x86 架构中，除零是异常情况，会下陷到操作系统进行处理；而在 AArch64 架构中，除零的结果还是零，被视为有效计算。在设计操作系统时，应当考虑到此类硬件设计细节上的区别。

3.2.3 案例分析：ChCore 启动与异常向量表初始化

由于异常向量表定义了操作系统的入口，因此异常向量表的初始化是启动后立刻要做的重要步骤。此外，操作系统在启动过程中还需要完成许多系统全局状态的初始化，为操作系统和应用程序的执行准备好运行环境。完成这些工作后，操作系统便能切换到用户态运行应用程序了。本节将以 ChCore 为例，介绍 AArch64 架构下启动过程中可能涉及的系统 ISA，以及如何对异常向量表进行初始化。

在计算机启动时，CPU 执行的第一段代码其实并非 ChCore 的代码。以在树莓派上运行 ChCore 为例，在加电后，主板中的固件和 SD 卡中的 bootloader 将先后被加载到内存并运行，进行基本的初始化工作。之后，ChCore 的二进制文件将被 bootloader 加载到内存中约定好的位置，这时才从头开始执行 ChCore 的代码。

如代码片段 3.5 所示，位于 ChCore 起始位置的代码为 `_start`，这便是进入 ChCore 时执行的第一段代码。在多核机器中，所有核心都会同时开始执行 `_start`，ChCore 需要选择某个 CPU 核心作为主要核心（通常是第一个）来初始化操作系统，同时其他核心将被暂时阻塞。AArch64 为我们提供了 MPIDR 这个系统寄存器来获取当前 CPU 核心的编号。通过系统指令 `mrs x8, mpidr_el1` 来读取 MPIDR 的值，ChCore 便能根据情况采取不同的启动逻辑。

代码片段 3.5 ChCore 的启动代码中对异常向量表的初始化

```

1 _start:
2     mrs x8, mpidr_el1
3     and x8, x8, #0xFF
4     cbz x8, primary
5
6     /* Code for secondary core */
7     ...
8     /* Init exception vector */
9     bl set_exception_vector
10    ...
11
12 primary:
13    /* Code for primary core */
14    /* init UART, Virtual Memory mapping in C */
15    ...
16    /* Init exception vector */
17    adr x0, ell_vector
18    msr vbar_el1, x0
19    ...
20
21 /* Exception Table */
22 ell_vector:
23    /* entry for other type of exception */
24    ...
25
```



```

26  /* entry for synchronous exception from EL0 */
27  .align 7      //128 bytes for each entry
28  b sync_el0_64
29
30  /* entry for interrupt from EL0 */
31  .align 7      //128 bytes for each entry
32  b irq_el0_64
33
34  /* entry for other type of exception */
35  ...

```

小思考

为什么操作系统内核的启动代码要用汇编语言而非 C 语言编写？

系统 ISA 的许多指令（比如读取 MPIDR）在 C 语言中并没有对应的语句，因此只能用汇编语言编写。另一个原因是，C 语言所依赖的运行时栈在系统刚启动的阶段尚未建立起来，SP 寄存器也没有初始化，所以现阶段还无法运行 C 语言的函数调用。

随后，CPU 的主要核心将执行操作系统的一系列初始化工作，例如对外设的访问、虚拟内存映射、初始化异常向量表等，本节主要关注其中对异常向量表的初始化。在上述代码中，异常向量表存放在 `el1_vector` 代表的位置。在 AArch64 中，异常向量表中的每一项都可以存放 128 字节的指令，但 ChCore 只存放了一条跳转指令，用于跳转到不同异常事件对应的处理函数。例如，对于来源于用户态的同步异常，内核将跳转到 `sync_el0_64`，并在其中保存通用寄存器，根据异常的详细信息进行不同的处理；对于异步异常（即中断），内核将跳转到 `irq_el0_64`，并执行相应的处理。

在编译 ChCore 时，异常向量表的内容已按照 AArch64 的布局在二进制文件中构造好了。在启动阶段，异常向量表便作为二进制文件的一部分被 `bootloader` 直接加载到内存中。因此，进入内核后，ChCore 只需要将异常向量表的起始地址放入指定的系统寄存器，便可完成异常向量表的初始化工作。上述代码中，ChCore 将 `el1_vector` 代表的异常向量表起始地址，通过 `msr` 指令存储到 `VBAR_EL1` 寄存器中。之后发生异常事件时，处理器便能跳转到 `VBAR_EL1` 寄存器指向的异常向量表中对应的表项，并跳转到操作系统内核进行处理。

3.2.4 用户态与内核态的切换

在用户态与内核态的切换过程中，有许多任务需要完成。这些任务大致归为两类：一类是保存用户程序的状态，一类是准备操作系统的运行环境。其中，需要保存的状态

主要是用户程序与操作系统共同使用、可能被操作系统覆盖的处理器状态。

准备操作系统的运行环境则包含更加复杂的操作。第一，为了处理异常事件，需要准备许多与异常事件相关的信息，例如异常事件的种类、触发事件的指令地址等；第二，由于操作系统所使用的栈不同于应用程序的栈，因此需要找到并切换至操作系统使用的栈；第三，根据不同的异常事件找到对应的异常处理函数地址，并跳转过去执行。不同处理器通常为每种异常情况分配好了编号[⊖]，根据异常编号，处理器便能自动在前文提到的异常向量表中找到对应的处理程序并进行跳转。

上述任务是由处理器与操作系统协同完成的。出于对通用性和复杂性的考虑，处理器只完成其中的必要步骤，剩余的步骤则需要操作系统来完成。接下来我们将以AArch64为例，介绍当异常发生时处理器与操作系统是如何协同实现特权级切换的。图3.12对切换过程中处理器状态中的软硬件分工进行了总结，我们先关注其中由处理器硬件完成的部分。

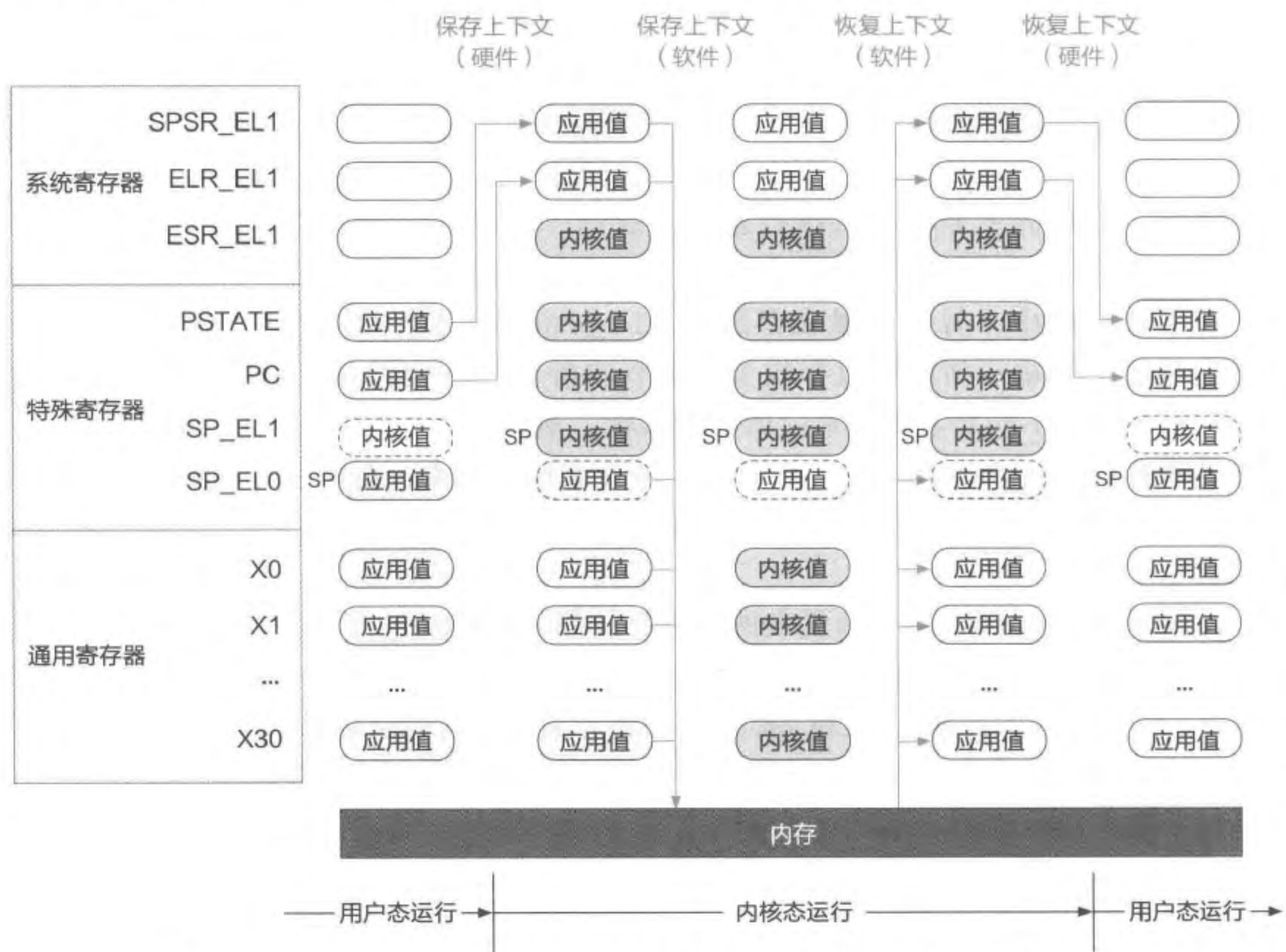


图 3.12 AArch64 架构下用户态 / 内核态切换过程中处理器状态的变化

⊖ 处理器的设计者同时也会保留一些编号，交由操作系统来分配。

处理器在特权级切换过程中的任务

在 AArch64 中，当从 EL0 切换到 EL1 时，处理器会进行如下操作。

1. 将发生异常事件的指令地址（即 PC 寄存器的值）保存在 ELR_EL1（Exception Link Register，异常链接寄存器）中。
2. 将异常事件的原因（例如，是执行 svc 指令导致的，还是访存缺页导致的）保存在 ESR_EL1（Exception Syndrome Register，异常症状寄存器）中。
3. 将处理器的当前状态（即 PSTATE 的值）保存在 SPSR_EL1（Saved Program Status Register，已保存的程序状态寄存器）中。
4. 保存与特定异常相关的信息，如缺页异常时，将引发异常的内存地址（后续章节将介绍）保存在 FAR_EL1（Fault Address Register，错误地址寄存器）中。
5. 栈寄存器不再使用 SP_EL0（用户态栈寄存器），开始使用 SP_EL1（内核态栈寄存器，需要由操作系统提前设置）。
6. 修改 PSTATE 寄存器中的特权级标志位，设置为内核态（EL1）。
7. 根据 VBAR_EL1 寄存器中保存的异常向量表基地址，以及发生异常事件的类型，找到异常处理函数的入口地址，并将该地址写入 PC，开始运行操作系统的代码。

可以看到，处理器的这些操作都是必要的。例如，PC 寄存器的值必须由处理器保存，否则当操作系统开始执行时，PC 将被覆盖。同样，栈的切换也必须由硬件完成，否则操作系统有可能使用用户态的栈，导致安全问题。

小思考

为什么操作系统不能直接使用应用程序在用户态的栈呢？

为了操作系统执行时的安全。应用程序的栈是用户态代码可以读写的，内核的数据不允许用户态的代码访问。

从内核态返回用户态的过程与上述过程是类似的。当操作系统处理完异常事件后，会执行一条特殊的汇编指令——eret[Ⓓ]，此时处理器会完成以下步骤。

1. 将 SPSR_EL1 中的处理器状态写入 PSTATE 中，处理器状态也从 EL1 切换到 EL0。
2. 栈寄存器不再使用 SP_EL1，开始使用 SP_EL0。注意 SP_EL1 的值并没有改变，所以下一次下陷时，操作系统依然会使用这个内核栈。
3. 将 ELR_EL1 寄存器中保存的地址写入 PC 中，并执行应用程序中的代码。

Ⓓ eret 的全称是 Exception RETurn。

操作系统在特权级切换过程中的任务

在处理器自动完成上述状态保存与恢复的基础上，操作系统还需要进一步将属于应用程序的 CPU 状态保存到内存中，用于之后恢复应用程序继续运行。应用程序需要保存的运行状态称为处理器上下文（Processor Context），它是应用程序在完成切换后恢复执行所需的最小处理器状态集合。联系 3.1 节对处理器状态的分类，处理器上下文中的寄存器具体包括：

- 通用寄存器 X0 ~ X30。应用程序和操作系统在运行时都需要使用这些寄存器。
- 特殊寄存器，主要包括 PC、SP 和 PSTATE。由于 PC 和 PSTATE 已经被 CPU 保存在额外的寄存器中，而 SP 寄存器在 CPU 中存在多份，因此操作系统不用将它们保存在内存中——除非操作系统决定切换到另一个应用而不是返回发生下陷的应用，才需要保存。
- 系统寄存器，包括页表基地址寄存器等（后文介绍）。由于这些寄存器无法由用户程序操作，不直接包含需要保存的用户程序状态，因此和前者类似，也不用保存——仅在操作系统切换应用时需要保存。

从上面的分析可以看出，在操作系统开始执行具体的异常处理函数之前，只需要保存通用寄存器，之后便可以根据异常的类型和来源进行针对性的处理了。同样，在返回用户态之前，操作系统也需要将之前保存的值重新写入通用寄存器。需要注意的是，在具体实现中应用程序的上下文会包含更多的内容，我们将在之后的章节中进一步介绍。

3.2.5 系统调用

系统调用是操作系统提供给应用程序的接口。例如，`printf` 函数就是通过 `write` 系统调用来实现的。系统调用是一种用户程序主动触发的异常，与一般的异常不同，系统调用需要在用户态与内核态之间传递一些额外的信息，例如用户程序提供的参数，以及操作系统给出的返回值。

对应用程序来说，系统调用与普通的函数调用看上去十分类似，区别在于：普通的被调用者与调用者都在用户态，共用一个栈；而系统调用的调用者在用户态，被调用者在内核态，两者使用不同的栈。因此系统调用的参数和返回值等信息，需要遵守一套与普通函数调用不同的调用惯例。然而，由于不同操作系统的系统调用以及调用管理往往不同，在应用程序中直接使用系统调用会使程序难以移植。为了解决不同系统不一致的问题，一些可移植操作系统接口标准逐渐发展起来，例如我们在第 1 章中提到过的 POSIX。我们平时编程时使用的各种库（如标准 C 库）会对系统调用做一层封装并处理部分错误情况，以进一步方便程序员使用。接下来我们将以 `write` 为例展示系统调用的流程。

图 3.13 以在 AArch64 上实现一个简单的 `write` 系统调用为例，展示了异常处理的过程。当用户程序调用 `printf` 时，实际上会使用标准 C 库来调用 `write` 系统调用。

根据操作系统的调用规约，在本例中，系统调用编号将放置在 X0，系统调用的参数则放置在 X1 ~ X3 寄存器中（不同的操作系统可能会使用不同的寄存器）。其中 X1 寄存器存储 I/O 的目的文件，常数 1 表示标准输出，即向终端（屏幕）输出文字；X2 寄存器存放需要输出字符串的首地址；X3 寄存器存放字符串的长度。

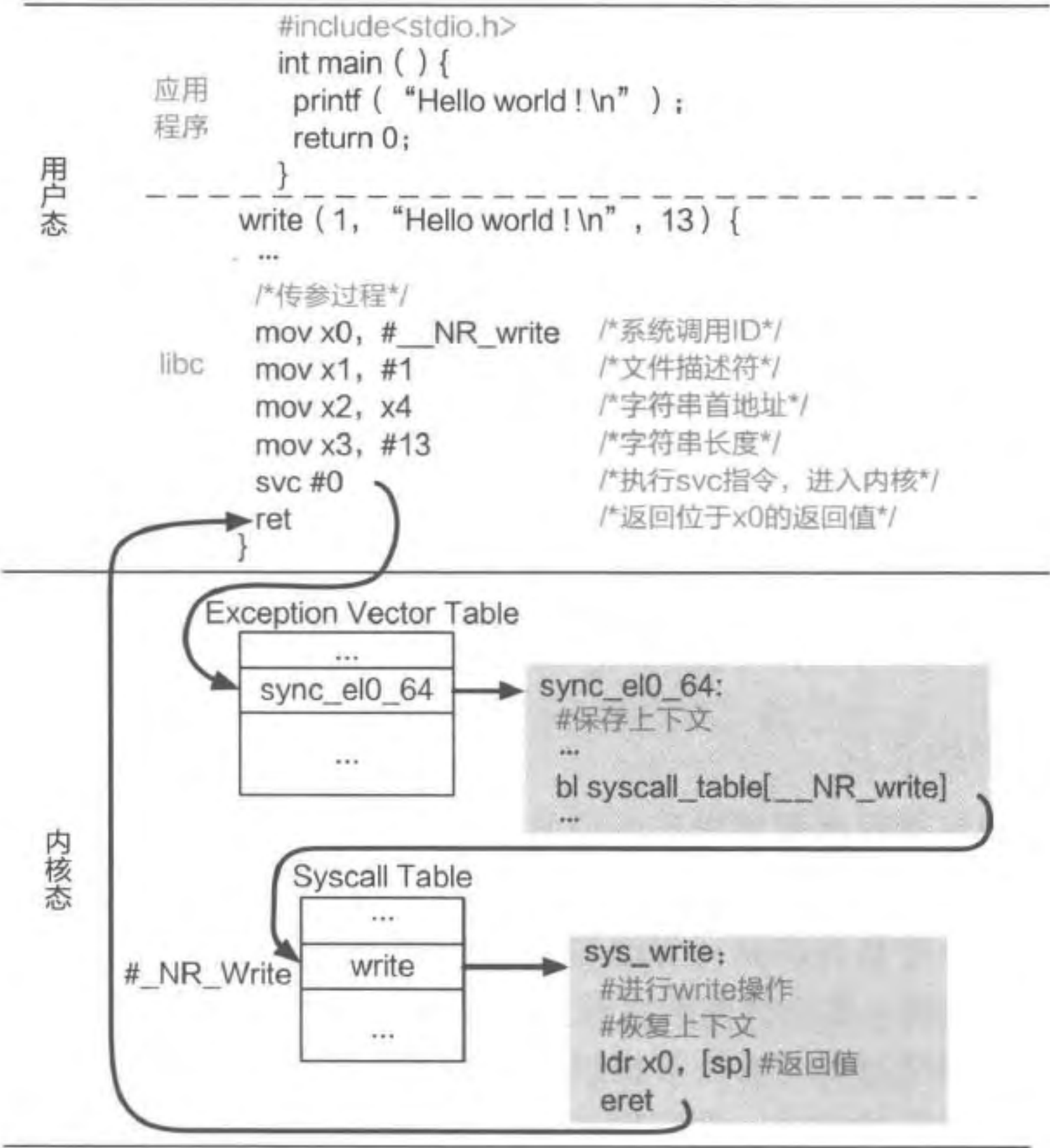


图 3.13 AArch64 架构下通过 svc 进行系统调用

在执行了 `svc` 后，处理器下陷到内核态，根据异常向量表找到对应的处理函数 `sync_el0_64` 并开始执行。异常处理函数在保存上下文后，会从保存在寄存器 `ESR_EL1` 中的异常原因得知这次异常是由 `svc` 指令导致的，从而根据用户保存在 X0 寄存器中的系统调用编号进入对应的系统调用^①，并根据寄存器中系统调用的参数向指定文件输出字符串^②。

最终，与函数调用类似，系统调用将返回值存放在调用规约规定的 X0 寄存器中，并通过 `eret` 从内核态返回用户态。

① 虽然 `svc` 指令可以传递一个常数参数，但本例并未使用这一传参方法。

② 由于虚拟内存（后文将介绍）的存在，内核态与用户态使用的内存存在一定形式的隔离。因此内核需要将 X2 寄存器中的字符串地址转换为内核能访问的地址，才能从应用程序的内存中获取字符串的内容。

小知识：如何观察应用程序的系统调用

除了 `write` 以外，我们平时编写的程序实际上已经用到了许多其他的系统调用。我们可以通过 `strace` 工具来查看一个程序在运行过程中使用了哪些系统调用。这里我们依然以前文的 `hello` 程序为例来查看它使用过的系统调用（见输出记录 3.1）。可以看到，在 Linux 的 shell 中执行这段 `hello` 程序时，首先 `execve` 系统调用被用来从文件系统中加载 `hello` 程序。此后，这段程序通过 `write` 系统调用来向终端输出字符串并最终显示到屏幕上。在这里，1 表示标准输出，13 则表示一共写了 13 个字符。这段程序执行完毕后，通过 `exit_group` 系统调用来退出自己。

输出记录 3.1 `hello` 程序使用的系统调用

```
[user@osbook ~] $ strace ./hello
execve("./hello", ["./hello"], 0x7ffc2e7d60 /* 28 vars */) = 0
...
write(1, "Hello World!\n", 13Hello World!)      =
13
exit_group(0)                                   = ?
...
```

3.2.6 系统调用的优化

系统调用作为应用程序调用操作系统的入口，其性能也非常重要。然而，不同于传统的函数调用，系统调用的过程复用了异常机制，因此不可避免地需要执行特权级切换、上下文保存等操作，导致其时延比普通的函数调用高 1 到 2 个数量级。对于需要频繁进行系统调用的应用来说，这是很大的性能开销。

那么，怎么才能绕过费时的异常机制来实现系统调用呢？可以通过在用户态和内核态之间共享一小块内存的方式，在应用与内核之间创建一条新的通道。具体来说又可以分为两种方法。

第一种方法是内核将一部分数据通过只读的形式共享给应用，允许应用直接读取。例如，`gettimeofday` 是一个常用的系统调用，用于获取当前的时间，可精确到微秒。传统的实现是：内核内部维护一个变量，以微秒为单位记录当前时间，在每次处理硬件时钟中断的时候更新这个变量；当发生系统调用时，内核读取并返回这个变量的值。为了绕过异常机制，内核可以将这个变量放在一个特殊的内存页中，这个内存页对应用来说是可读但不可写的。在 `libc` 中，对 `gettimeofday` 的实现仅仅是读取这个变量，然后返回变量的值，而不需要陷入内核，从而大大降低了时延。对于应用程序来说，由于无法修改这个值，所以也不会影响系统整体的安全性。Linux 的 `vDSO`（`virtual Dynamic Shared Object`）机制就是基于这种方法。这种方法的缺点在于，如果系统调用需要修改内核中的变量，或者在运行过程中需要读取更多内核数据，该方法就不适用了。

第二种方法则是允许应用以“向某一块内存页写入请求”的方式发起系统调用，并通过轮询来等待系统调用完成。内核同样通过轮询来等待用户的请求，然后执行系统调用，并将系统调用的返回值写入同一块内存页以表示完成。但应用和内核怎么能同时轮询呢？这个设计的关键点在于：让内核独占一个 CPU 核心，这个核心一直在内核态运行，而其他 CPU 核心则一直在用户态运行。这样，从系统整体来看，对于任何一个 CPU 核心，都不会发生从用户态到内核态的切换。这种设计可以大大降低系统调用的时延：在应用将请求写入内存页后的下一个时钟周期，处于轮询状态的内核立即可以读到这个请求，并开始运行处理函数；同样，当内核将返回结果写入内存页后，在另一个 CPU 核心处于轮询状态的应用立即可以读到结果并继续运行。FlexSC^[2]使用了这种方法。

然而，第二种方法同样存在缺点。第一个缺点在于，如果某个应用发起系统调用请求的时候，内核正在处理上一个系统调用，则时延依然会很长；如果有多个应用同时发起请求，内核需要一个个顺序处理，则时延可能比原来更长。导致这个问题的原因在于内核没能充分使用多核。解决方法也很直接：让多个 CPU 核心同时运行在内核态并轮询用户的请求。核心的数量可以根据具体的负载确定：当内核忙不过来时，占用的核心多一些，反之则少一些。例如，假设主机一共有 16 个 CPU 核心，当 1 个核心运行内核态不够时，可以增加到 2 个或更多核心专门运行内核代码，剩下的 14 个则一直运行用户态代码。

第二个缺点在于，如果整个系统只有一个 CPU 核心，那么该怎么办？一种直观的方法是将轮询改成批处理。当 CPU 运行在用户态时，应用程序一次发起多个系统调用请求（这里可以基于用户态的调度实现用户线程的切换），同样将请求和参数写入共享内存页。然后 CPU 切换到内核态，内核一次性将所有系统调用处理完，把结果写入共享内存页，再切换回用户态运行。这种方法的好处并不在于时延（事实上对于每个系统调用来说，时延反而可能变长），而是在于系统的整体吞吐率：对于系统整体来说，由于特权级的切换次数变少了，所以在单位时间内的有效工作量增加了。

3.3 操作系统提供的基本抽象与接口

本节主要知识点

- 为什么需要进程这一抽象？
- 相比直接使用物理内存地址，使用虚拟内存地址有哪些优势？
- 为什么说“一切皆文件”？

基于异常机制和系统 ISA，操作系统为应用程序提供底层硬件的抽象与接口，使应